How Robust Are Exception Handling Defect Models? A Replication of de Pádua and Shang (MSR 2018)

Group 9: Emmanuel Adegboye, Benjamin Emeshili, Marco Spensieri

***Abstract*.** De Pádua and Shang reported that some exception handling anti-patterns, such as Dummy Handler and Generic Catch, are linked to post-release defects. Their models depend on several fixed analysis settings. We will reproduce the study from the authors' replication package and then vary five of those settings, one at a time, to see which findings hold. A first run already reproduces the base and anti-pattern models, and it shows that the published scripts cannot produce the paper's exception flow results.

I. Overview and Motivation

Exception handling code runs when things go wrong, so mistakes in it tend to surface as failures after release. Teams maintaining large systems have limited time for cleanup. If certain handling habits really are tied to defects, those habits are a sensible place to spend refactoring effort.

De Pádua and Shang [1] studied this question on Hadoop, Hibernate (Java) and Umbraco (C#). They built logistic regression models of whether a file has a post-release defect and found that a few anti-patterns explain defects beyond size, churn and ownership. Before fitting, they made several modeling choices: a correlation cutoff of 0.7, a redundancy cutoff of $R^2 = 0.9$, a budget of one predictor per 15 files, a $\log10(x + 1)$ transform, and backward selection at $p < 0.05$. Prior work shows that choices like these can change which metrics a defect model calls important [4], [7]. If a finding only appears under one set of settings, it is weak guidance for maintainers. Replication is how the field checks this [9].

II. Research Questions and Scope

Original study. The paper asks whether exception flow characteristics (RQ1) and 17 exception handling anti-patterns (RQ2) add explanatory power to a base model of traditional metrics. It compares three models per project: BASE, BSFC (base plus flow metrics) and BSAP (base plus anti-pattern metrics). It reports that flow metrics help in the Java projects and that Dummy Handler, Generic Catch, Ignoring Interrupted Exception and Log and Throw are significant in at least one project.

Our questions.

- **RQ1 (Reproducibility):** Can the published package reproduce Tables 4 and 5 of the paper on a current R installation?
- **RQ2 (Sensitivity):** Do the significant exception handling metrics stay significant, with the same sign, when each modeling setting changes?
- **RQ3 (Robustness of the conclusion):** Does the gain in explanatory power of BSAP and BSFC over BASE survive those changes?

Scope and new study context. We keep the original three projects and their published data. Our new context is the analysis pipeline itself: we change the settings the authors fixed. We do not re-extract metrics, add projects, or change how defects were linked to files. The authors' Umbraco model is a good test case, because its correlation cutoff had to drop to 0.24 to fit the predictor budget.

III. Related Work

The original study builds on the authors' earlier work on the prevalence of exception handling anti-patterns [2] and on exception flow analysis [3]. Our extension draws on three papers about defect model construction. Jiarpakdee et al. [4] show that how correlated metrics are removed changes the ranking of important metrics, which motivates varying the correlation and redundancy cutoffs. AutoSpearman [5] offers an automatic alternative to a fixed cutoff that we may use as a reference point. Tantithamthavorn et al. [6] compare model validation techniques and support keeping the optimism-reduced bootstrap. Shepperd et al. [7] found that analysis choices explain much of the variation between published defect prediction results. The modeling steps follow Harrell [8], whose guidance on predictor budgets we revisit. These studies look at defect models in general. We apply the same concern to one published finding about exception handling and test it on that study's own data. In the terms of Shull et al. [9], ours is a close replication on the original data, extended with a sensitivity analysis.

IV. Methodology

Data and tools. We use the authors' replication package: five file-level metric files covering 10,195 source files, two R notebooks, two helper scripts and their saved outputs. The analysis runs in R 4.6.1 with the rms, Hmisc and caret packages.

Phase 1: Reproduce. We run both notebooks and compare our outputs with the authors' saved output files and with Tables 4 and 5. A result counts as reproduced if the optimism-reduced R^2 is within 0.02 and the same metrics are significant with the same sign. We record every error, fix and version in a reproducibility log.

Phase 2: Vary the settings. We change one setting at a time and keep the rest at their original values (Table I). This gives nine configurations per model, or 81 model fits across three models and three projects. We add a fixed random seed so bootstrap results repeat exactly.

TABLE I
Settings varied in the sensitivity analysis

Setting	Original	Values tested
Correlation cutoff	0.7	0.6, 0.8
Files per predictor	15	10, 20
Redundancy cutoff (R^2)	0.9	0.8
Transformation	$\log10(x + 1)$	none
Selection threshold (p)	0.05	0.01, 0.10

Phase 3: Classify findings. For each metric that was significant in the original models, we count the configurations in which it stays significant with the same sign. A metric is *robust* if it holds in all nine, *moderately robust* in five to eight, and *fragile* in four or fewer. We also report the range of R^2 for each model.

Evaluation measures. For RQ1, we report the absolute difference in optimism-reduced R^2 and the overlap between our set of significant metrics and the paper's. For RQ2, each originally significant metric gets a stability score: the share of the nine configurations in which it stays significant with the same sign. For RQ3, we compute the gain over BASE (BSAP minus BASE, BSFC minus BASE) in every configuration and check whether it stays positive.

Validation. A fixed seed makes every run repeatable. Before trusting any variation, we confirm that the original configuration still matches Phase 1. A second member then reruns the whole pipeline from our instructions and must get identical outputs.

Threats to validity. Changing one setting at a time misses interactions between settings. We inherit any errors in the original metric extraction and defect linking. Results cover only three projects. Our cut-offs for robust and fragile are our own choice, so we also report the raw stability scores.

Planned adaptations. Three changes to the original pipeline are needed. First, one line in the helper scripts fails on R 4.3 and later; we replace it with an equivalent check. Second, the predictor budget is hard-coded in four places and the redundancy cutoff relies on a function default, so we move all five settings into one configuration file. Third, we correct the exception flow model, as described next.

Preliminary feasibility check. We have already run the full original pipeline. The data preparation step reproduced the authors' merged dataset exactly. The BASE and BSAP models matched the paper (Table II), and the same anti-patterns came out significant. Bootstrap-based values differed by up to 0.023 because the code sets no random seed. The BSFC model did not match. In our output and in the authors' own saved output, every BSFC row is a copy of the BSAP row. The cause is a leftover line in the data reduction function that replaces its input with the BSAP data. We will rerun BSFC with that line removed and check it against the paper.

TABLE II
Optimism-reduced R^2 (paper / our first run)

Project	BASE	BSAP	BSFC
Hadoop	0.33 / 0.33	0.35 / 0.35	0.37 / 0.35
Hibernate	0.21 / 0.21	0.23 / 0.23	0.28 / 0.23
Umbraco	0.09 / 0.09	0.18 / 0.18	0.09 / 0.18

V. Feasibility and Work Plan

Resources. All data and scripts are public and need no paid tools. The original metric extraction used a commercial tool, but we reuse its output. One full run of the original notebook takes about 25 minutes on a laptop. Nine configurations therefore need about four hours, which we can run overnight. At the end we publish a reproducibility package on GitHub with our scripts, the configuration file, the environment record and all outputs.

Risks and alternatives. If the corrected BSFC model still differs from the paper, we report the gap as a reproducibility finding and limit RQ2 and RQ3 to BASE and BSAP. If Umbraco is too small for some settings, we report those configurations as not estimable. If the package goes offline, we keep an archived copy. If time runs short, we drop the redundancy and transformation variations first.

Timeline. A1: this proposal. A2: full reproduction, corrected BSFC, reproducibility log and related work. A3: sensitivity runs for Hadoop and Hibernate. A4: Umbraco, classification of findings and an independent rerun by a second member. Presentation and final report follow.

Responsibilities.

Emmanuel Adegboye: Coordination and submissions, validation rerun and the reproducibility package

Benjamin Emeshili: R environment and reproduction.

Marco Spensieri: sensitivity runs and robustness tables.

All members review each milestone before submission.

References

[1] G. B. de Pádua and W. Shang, "Studying the relationship between exception handling practices and post-release defects," in *Proc. 15th Int. Conf. Mining Software Repositories (MSR)*, 2018, doi: 10.1145/3196398.3196435.

[2] G. B. de Pádua and W. Shang, "Studying the prevalence of exception handling anti-patterns," in *Proc. 25th Int. Conf. Program Comprehension (ICPC)*, 2017.

[3] G. B. de Pádua and W. Shang, "Revisiting exception handling practices with exception flow analysis," in *Proc. 17th Int. Working Conf. Source Code Analysis and Manipulation (SCAM)*, 2017.

[4] J. Jiarpakdee, C. Tantithamthavorn, and A. E. Hassan, "The impact of correlated metrics on the interpretation of defect models," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 320–331, 2021.

[5] J. Jiarpakdee, C. Tantithamthavorn, and C. Treude, "AutoSpearman: Automatically mitigating correlated software metrics for interpreting defect models," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2018.

[6] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 43, no. 1, pp. 1–18, 2017.

[7] M. Shepperd, D. Bowes, and T. Hall, "Researcher bias: The use of machine learning in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 40, no. 6, pp. 603–616, 2014.

[8] F. E. Harrell, *Regression Modeling Strategies*, 2nd ed. Springer, 2015.

[9] F. J. Shull, J. C. Carver, S. Vegas, and N. Juristo, "The role of replications in empirical software engineering," *Empirical Softw. Eng.*, vol. 13, no. 2, pp. 211–218, 2008.